Studio Nx

AI 구현 설명 16 — 회원가입 API 템플릿

2026-10-12

AI 구현 설명 16 — 회원가입 API 템플릿

항목	내용
작성	Claude (AI)
작성일	2026-10-03
작업	P1-04 회원가입 API + 이메일·닉네임 중복 확인 (**템플릿**), 이전 문서: 08_backend_core.md
브랜치	`feature/phase1/p1-04-signup-api`
원칙	백엔드는 **템플릿만** 만들었습니다 (라우터 연결, 요청과 응답 스키마, 함수 이름과 TODO). 실제 구현은 사용자가 합니다

---

1. 무엇을 만들었나

파일	상태	내용
`app/api/v1/auth.py`	템플릿	`POST /auth/signup`, `GET /auth/check-email`, `GET /auth/check-nickname` — 경로, 상태 코드, 응답 모양은 완성했고 본문은 TODO입니다
`app/api/v1/router.py`	수정	`auth.router`를 `/auth`로 연결했습니다
`app/schemas/auth.py`	완성과 TODO	`SignupRequest`, `SignupResponse`, `AvailabilityResponse`, `NICKNAME_MIN/MAX`가 있고, 이메일 형식 검사만 TODO입니다
`app/services/auth_service.py`	템플릿	`normalize_email`, `check_email`, `check_nickname`, `release_withdrawn`, `signup`의 처리 순서를 TODO로 적었습니다
`tests/test_auth_routes.py`	완성	세 주소가 `/api/v1/auth`에 연결되었는지 확인합니다 (구현 전에도 통과합니다)

> 이미 있던 `core/security.py`의 `check_password_rules`, `hash_password`(P1-04 TODO)도 이번 작업에서 채웁니다.

2. 왜 이렇게 했나

2.1 라우터 → 서비스로 나눔

층	하는 일
라우터 (`api/v1/auth.py`)	요청을 받고, 서비스의 결과를 응답 스키마로 바꿉니다
서비스 (`services/auth_service.py`)	DB 조회, 규칙 검사, 저장, `AppError` 던지기를 맡습니다

`check_nickname`은 회원 정보 수정(P1-08)에서도 그대로 다시 씁니다.

2.2 API 모양

Method	Path	성공	실패
POST	`/auth/signup`	201 `{user_id, email, nickname}`	409 `EMAIL_TAKEN` / `NICKNAME_TAKEN`, 422 `VALIDATION_ERROR`

Method	Path	성공	실패
GET	`/auth/check-email?email=`	200 `{"available": true}` 또는 `{"available": false, "reason": …}`	422 (형식 오류, 선택)
GET	`/auth/check-nickname?nickname=`	200 `{"available": false}`	422 (길이 초과)

- 화면(`join.js`)이 보내는 JSON `{ email, nickname, password }`와 이름을 맞췄습니다.
- 응답에는 비밀번호와 해시를 넣지 않습니다 (PRD 01 §5).

2.3 검사 위치

검사	위치	이유
닉네임 2~20자, 길이 상한	스키마 `Field`	FastAPI가 자동으로 422로 응답합니다
비밀번호 조건 6가지	`check_password_rules` (서비스에서 호출)	실패한 조건 목록을 `details.password`에 담아서, 화면이 어떤 조건이 틀렸는지 보여 줄 수 있게 합니다
이메일과 닉네임 중복	서비스	DB 조회가 필요하기 때문입니다
동시 가입	`db.commit()`의 `IntegrityError`	중복 확인 직후에 다른 사람이 같은 값으로 가입할 수 있으므로, DB의 UNIQUE가 최종 방어선입니다

2.4 탈퇴 후 재가입 (2026-10-03 결정)

그 이메일이나 닉네임을 가진 행	결과
없음	사용할 수 있습니다
`ACTIVE`, `SUSPENDED`	사용 중이며, 409 `EMAIL_TAKEN` / `NICKNAME_TAKEN`으로 응답합니다
`DELETED`, 탈퇴한 지 3일 안	409 `REJOIN_TOO_SOON`과 `available_at`(다시 쓸 수 있는 시각)으로 응답합니다
`DELETED`, 3일이 지남	`release_withdrawn`으로 예전 행의 값을 `deleted+{id}@withdrawn.invalid`, `deleted_{id}`로 바꾼 뒤 가입시킵니다

- `email`과 `nickname`은 UNIQUE라서, 탈퇴한 행이 값을 쥐고 있으면 새 행을 넣을 수 없습니다. 그래서 다시 쓰려는 사람이 나타난 시점에 예전 행의 값을 비웁니다. DB 구조는 바꾸지 않습니다.
- 기다리는 기간은 `REJOIN_WAIT_DAYS = 3` 한 곳에서 정합니다. 닉네임도 같은 규칙을 적용해, 탈퇴 직후 다른 사람이 같은 이름을 쓰지 못하게 했습니다.
- 중복 확인 응답에 `reason`, `available_at`을 더해 화면이 "Oct 6 이후 다시 가입할 수 있어요"처럼 안내할 수 있게 했습니다.

2.5 이메일 정규화

`normalize_email`(앞뒤 공백을 지우고 소문자로 바꿈)을 중복 확인과 저장에 모두 거치게 했습니다. `A@x.com`과 `a@x.com`이 서로 다른 계정이 되는 것을 막기 위해서입니다.

3. 검증 결과 (2026-10-03)

확인	결과
`pytest`	7개가 통과했습니다 (기존 6개와 라우터 연결 1개)
`ruff check`	통과했습니다
OpenAPI	`/api/v1/auth/signup`, `/check-email`, `/check-nickname` 세 주소가 `/docs`에 보입니다

> 본문이 `NotImplementedError`라서, 구현 전에 이 주소를 부르면 500이 납니다.

4. 남은 일 (사용자)

순서	할 일	파일
1	`User` 모델 나머지 컬럼 (P1-03 TODO)	`app/models/user.py`
2	오류 처리기 등록 (P1-03 TODO). 하지 않으면 `AppError`가 500으로 나갑니다	`app/core/errors.py`
3	`check_password_rules`, `hash_password`	`app/core/security.py`
4	이메일 형식 검사 (`EmailStr` 또는 정규식)	`app/schemas/auth.py`
5	서비스 4개 함수	`app/services/auth_service.py`
6	라우터 3개 본문	`app/api/v1/auth.py`
7	`/docs`에서 직접 호출해 확인하고, 테스트를 추가합니다 (P1-18)	`tests/`

5. 자주 받을 질문 대비

- **Q. 비밀번호 규칙을 왜 스키마가 아니라 서비스에서 검사하나요?** 스키마 검사는 실패하면 FastAPI 기본 문구만 돌려줍니다. 서비스에서 `check_password_rules`를 부르면 실패한 조건 이름(`upper`, `special` 등)을 그대로 돌려줄 수 있어, 화면 조건 목록과 맞출 수 있습니다.
- **Q. 중복 확인을 했는데 왜 `IntegrityError`도 잡나요?** 확인과 저장 사이에 다른 요청이 끼어들 수 있습니다. 마지막 판단은 DB의 `uq_users_email`, `uq_users_nickname` UNIQUE 키가 합니다.
- **Q. 가입하면 바로 로그인되나요?** 아닙니다. 201을 받으면 화면이 `home.html#login`으로 이동해 로그인 모달을 엽니다. 토큰 발급은 P1-05 `/auth/login`만 합니다.
- **Q. `Query(...)`를 왜 `Annotated`로 썼나요?** FastAPI 권장 방식이고, 함수 기본값에 호출을 넣지 말라는 린트 규칙(B008)도 통과합니다.